8

Lisp から借用したアイデア

ブロックは、Ruby で最もよく使われる強力な機能のひとつだ。ブロックを使えば、`Enumerable` のメソッドにコード片を渡して実行できる。たとえば `each` や `detect`、`inject` などがそうだ。`yield` キーワードを使えば、独自のイテレータや、さまざまな目的に応じてブロックを呼び出す機能を用意できる。ブロックを活用して書かれた Ruby のコードは、同じ処理を C のような Ruby よりも古い言語で書いた場合よりも、わかりやすく、洗練された、表現力の高いものになることが多い。

だからといってブロックが新しいアイデアだと慌てて結論づけてはいけない！ブロックの登場は Ruby が最初、などということは全くない。ブロックの背後にはコンピュータサイエンスの概念が控えており、それはクロージャと呼ばれている。クロージャを最初に発明したのは Peter J. Landin で、1964 年のことだ。これは、John McCarthy が Lisp のオリジナルのバージョンを作成した 1958 年から数年後にあたる。クロージャはその後、Lisp で――もっと正確に言うなら Scheme で採用された。Scheme は Gerald Sussman と Guy Steele が 1975 年に発明した Lisp の方言だ。Sussman と Steele が Scheme でクロージャを活用したことによって、ブロックのようなアイデアが多くのプログラマに認知されることとなったのだ。

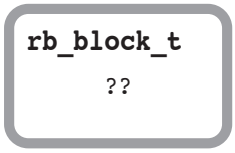
とはいうものの、クロージャという単語は本書の文脈では具体的にどんな意味があるのだろうか？ 別の言い方をするなら、Ruby のブロックとは正確には何なのか？ 単に `do` と `end` キーワードの間に書かれたコード片なのだろうか？この章では、Ruby がブロックを内部的にどう実装しているかを確かめながら、Sussman と Steele が 1975 年に作ったクロージャの定義をブロックがどう満たし

This document is licensed to kyomuwohakai@gmail.com.

ているのかを説明する。また、ブロックやラムダ、Proc のそれぞれが、クロージャというものを異なった観点で捉えているということも説明する。

ブロック：Ruby におけるクロージャ

Ruby は個々のブロックを rb_block_t という C 構造体で表現する。図 8-1 にこれを示す。Ruby が rb_block_t に何を格納しているかを学べば、ブロックとは何なのかを正確に理解できる。



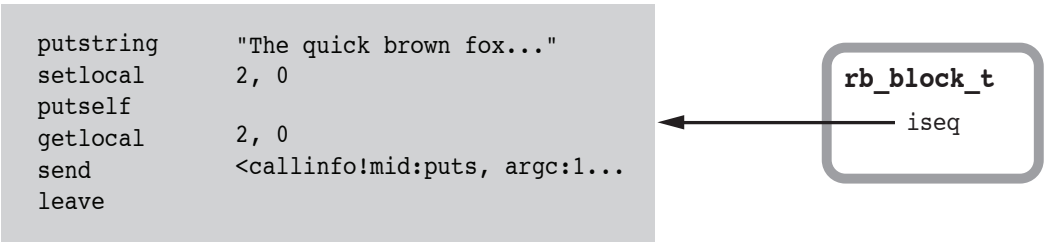
▲ 図 8-1　rb_block_t 構造体の中はどうなっている？

第 5 章で RClass 構造体に行ったのと同じように、私たちが知っている Ruby のブロックの機能から、rb_block_t の中身を推定していこう。まずはブロックの一番わかりやすい性質から始めることにする。ブロックは Ruby のコード片で構成されている。ということは、内部的にブロックは YARV のバイトコード命令の集まりだということだ。たとえばリスト 8-1 のように、ブロックをパラメータとして渡してメソッドを呼び出したとする。

▽ リスト 8-1　見た目には、ブロックはただの Ruby のコード片だ

```
10.times do
  str = "The quick brown fox jumps over the lazy dog."
  puts str
end
```

10.times を呼び出したとき、Ruby は反復させるコードを把握する必要がある。ということは、rb_block_t 構造体は繰り返し対象となるコードへのポインタを持たねばならない。これを図 8-2 に示す。



▲ 図 8-2　rb_block_t 構造体はコード片の YARV 命令列へのポインタを持つ

iseq は、ブロック中の Ruby コードを表現する YARV 命令列へのポインタだ。

これ以外にもブロックの特性として、明らかなのだが見落とされがちなものがある。それはブロックがその外側、つまり親のスコープの変数にアクセスできる

This document is licensed to kyomuwohakai@gmail.com.

ことだ。リスト 8-2 にその様子を示す。

▽ リスト 8-2　ブロック内のコードから、周囲のコードにある変数 str にアクセスする

```
① str = "The quick brown fox"
② 10.times do
③   str2 = "jumps over the lazy dog."
④   puts "#{str} #{str2}"
  end
```

④ の puts 呼び出しは、ブロック中の変数 str2 と ① でブロックの外側で定義されている変数 str を参照している。ご覧のとおり、ブロックは周囲のコードにある値にアクセスできる。この機能のおかげで、ブロックは有用なものになっているのだ。

ある意味、ブロックは二重人格だともいえる。一方では、ブロックは分離されたメソッドのように振る舞う。ブロックをメソッドのように呼び出すこともできれば、メソッドへの引数として渡すこともできる。また一方では、ブロックは自身を取り囲む関数やメソッドの一部であるかのようにも振る舞う。

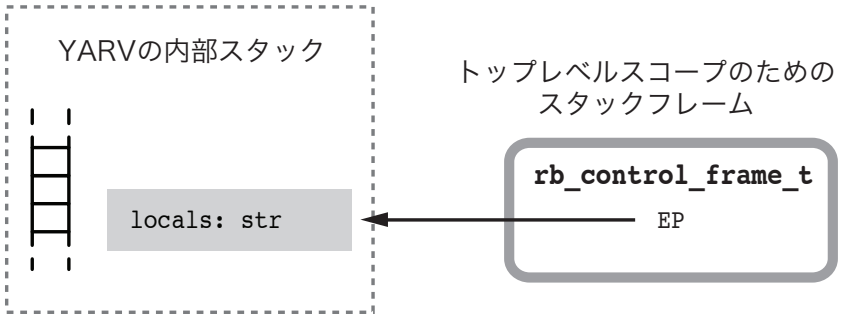
Ruby のブロック呼び出しを一歩ずつ確認する

Ruby の内部でブロックはどのように実行されるのだろうか？ YARV はブロックを、分離されたメソッドとして実装しているのだろうか？ それとも、ブロックを取り囲むメソッドの一部として実装しているのだろうか？ リスト 8-2 のコードを例にとって、ブロックを呼び出したときに Ruby の内側で何が起きるのかを一歩ずつ確認していくとしよう。

Ruby がリスト 8-2 の最初の行 ① にある str = "The quick brown fox"を実行すると、YARV はローカル変数 str を内部スタックに配置する。YARV は str の位置を rb_control_frame_t 構造体にある EP を使って追跡する。これを図に表したものが図 8-3 だ[*1]。

[*1] もし外側のコードが関数（メソッド）内に位置していたなら、図にあるように EP はスタックフレームにポイントされるだろう。けれど、もし外側のコードがトップレベルスコープに位置していた場合は、Ruby は代わりに TOPLEVEL_BINDING 環境に変数を保存するために動的アクセスを使う。どちらにしろ、EP は常に str 変数の居場所を示すということに変わりはない。

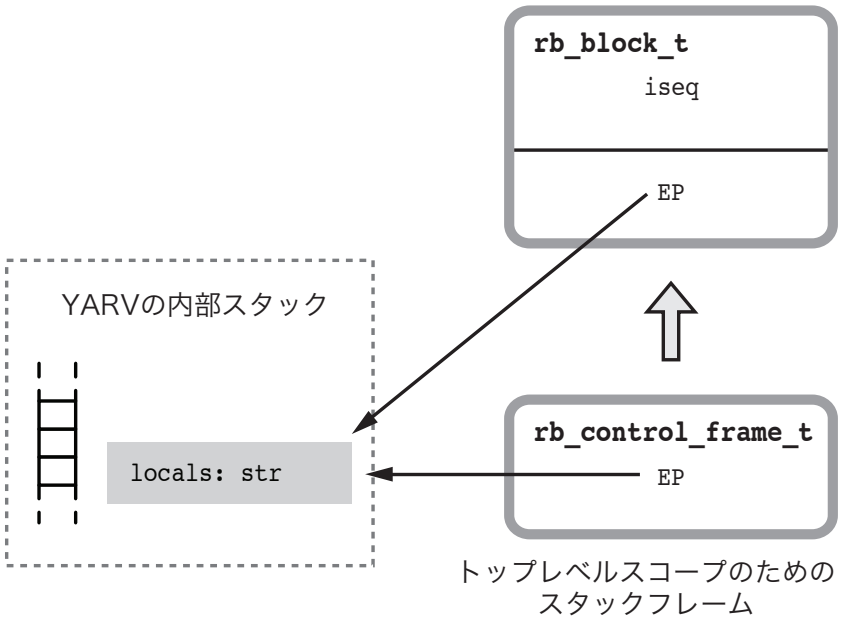
This document is licensed to kyomuwohakai@gmail.com.



▲ 図 8-3　Ruby はローカル変数 `str` をスタックに保存する

次に、Ruby はリスト 8-2 の ② で `10.times do` に到達する。反復処理を実際に実行する前に――つまり `times` メソッドを呼び出す前に――Ruby はブロックを表現する新しい `rb_block_t` 構造体を作成し、初期化する。ブロックは実際には単に `times` メソッドの引数なので、Ruby はこの時点でブロック構造体を作成する必要がある。図 8-4 に、この新しい `rb_block_t` 構造体を示す。

新しいブロック構造体を作成すると、Ruby は新しいブロックに `EP` の現在の値をコピーする。つまり、Ruby は新しいブロックに現在のスタックフレームの位置を保存する。

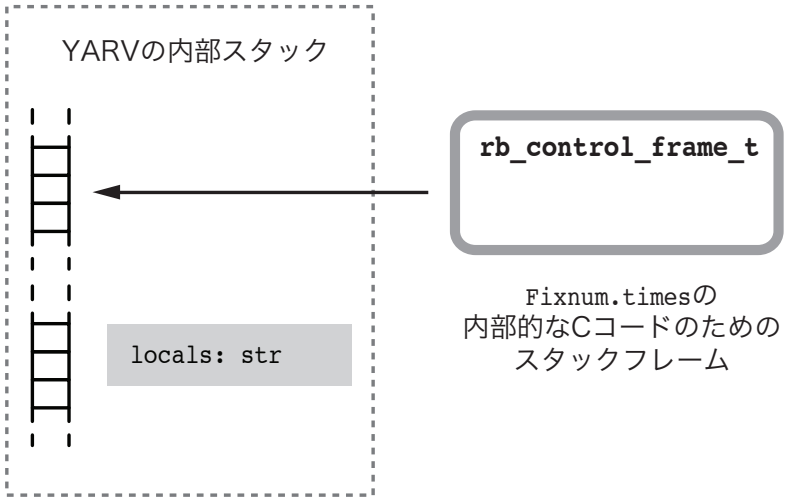


▲ 図 8-4　メソッドを呼び出してブロックを渡す前に、Ruby は新しい `rb_block_t` 構造体を作成する

次に、Ruby は `Fixnum` クラスのインスタンスであるオブジェクト `10` の `times` メソッドを呼び出す。メソッドを呼び出している間、YARV は内部スタック上に新しいフレームを作成する。この段階で 2 つのスタックフレームを持つことにな

This document is licensed to kyomuwohakai@gmail.com.

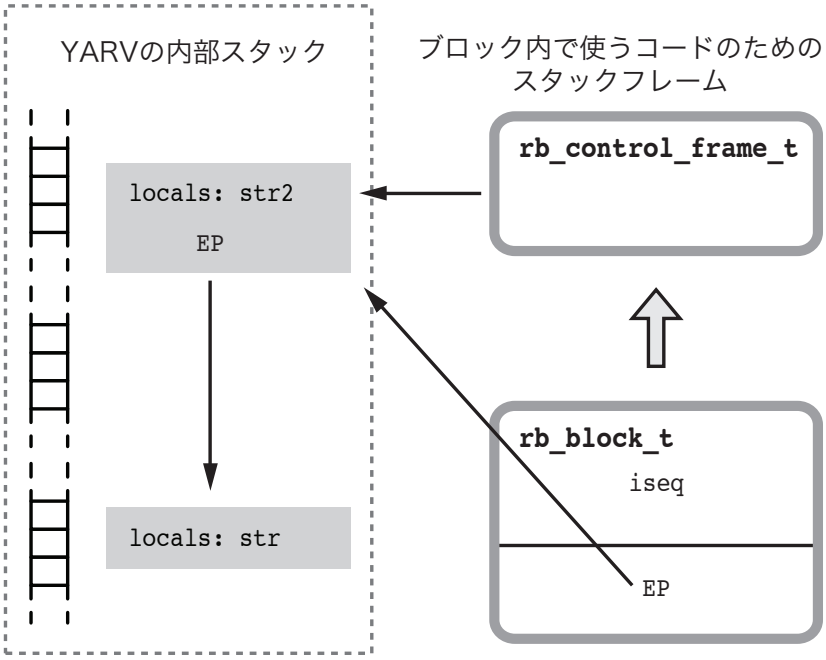
る。まず 1 つは Fixnum.times メソッド用の新しいスタックフレーム、そしてもう 1 つはトップレベル関数によって使われていた元のスタックフレームだ（図 8-5 参照）。



▲図 8-5　10.times を実行すると、Ruby は新しいスタックフレームを作成する

Ruby は内部的には C コードを使って times メソッドを実装している。これは組み込みメソッドだ。けれど、おそらく想像しているような実装になっているはずだ。Ruby は反復処理を開始すると、0、1、2 と 9 まで順に数をカウントアップしていきながら yield を呼び出し、整数それぞれに対して 1 度ずつブロックを呼び出す。最後に、yield を実装するコードはループを移動するたびに内部的に毎回ブロックを呼び出し、ブロック内で使うコード用にスタックの一番上に 3 つめのフレームをプッシュする。図 8-6 にこの 3 つめのスタックフレームを示す。

This document is licensed to kyomuwohakai@gmail.com.



▲ 図 8-6　Ruby は 10.times がブロックを実行するときに 3 つめのスタックフレームを作成する

図の左側に 3 つのスタックフレームがある。

- 一番上はブロック用の新しいスタックフレームで、リスト 8-2 の ③ で定義された変数 str2 を含んでいる。
- 真ん中は Fixnum#times メソッドを実装している内部の C コードによって使われるスタックフレームだ。
- 一番下は元の関数のスタックフレームでリスト 8-2 の ① で定義された変数 str を含んでいる。

新しいスタックフレームを作成する際、Ruby の内部の yield コードはブロックから新しいスタックフレームに EP をコピーする。これでブロック内部のコードはどちらのローカル変数にもアクセスできる。自身のローカル変数には直接 rb_control_frame_t 構造体を通して参照でき、親のスコープの変数には動的変数アクセスを使い EP を通して間接的に参照できる。具体的には、これによってリスト 8-2 の ④ で puts が親のスコープの str を参照することが可能になる。

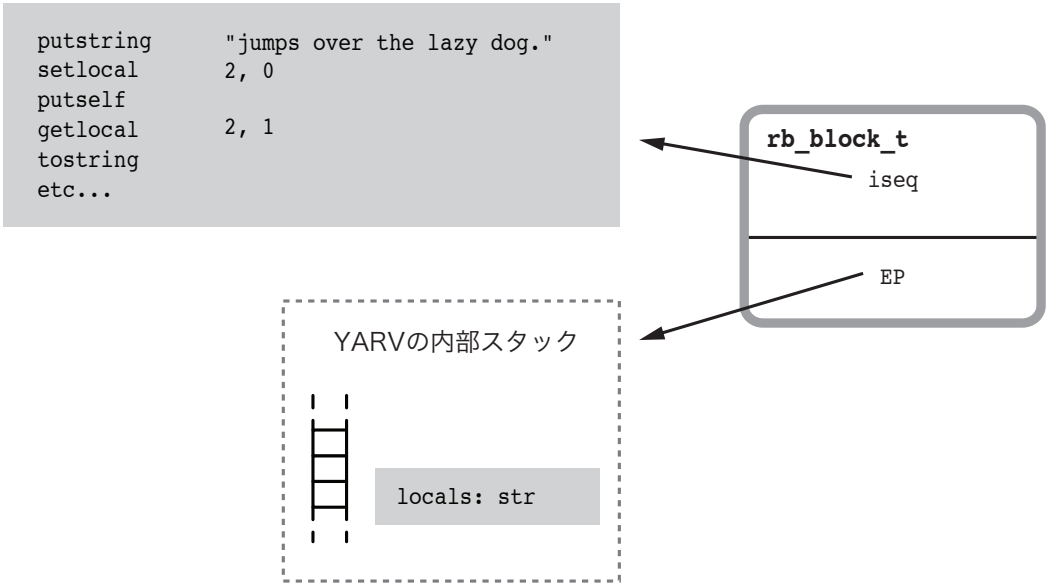
1975 年から借用したアイデア

ここまで、Ruby の rb_block_t 構造体には 2 つの重要な値が含まれていることを見てきた。

This document is licensed to kyomuwohakai@gmail.com.

- YARV コード命令のコード片へのポインタ――iseq ポインタ
- YARV の内部スタック、ブロックが作成されたときに一番上にあったスタックの位置へのポインタ――EP

図 8-7 に、rb_block_t 構造体中のこれら 2 つの値を示す。



▲図 8-7　Ruby のブロックが YARV 命令列のコード片へのポインタと、YARV スタック上の位置を示すポインタが含まれていることを見てきた

また、ブロックが周囲のコード中にある変数にアクセスするとき、Ruby が EP を使うことを確認した。一見、これはとても実装寄りで些細な話にも見える。これは Ruby のブロックに当然示してほしい振る舞いであるし、EP はブロックの内部実装のマイナーでつまらない部分のように見える。そうじゃないか？

IBM 704 は 1960 年代のはじめに登場した Lisp を実行する最初のコンピュータ（写真提供：NASA）

実際には、EP は Ruby 内でたいへん重要な部分だ。なぜなら、EP は Ruby によるクロージャ実装の基礎だからだ。クロージャとは、1990 年代に Ruby が登場するはるか以前に、Lisp に導入されたコンピュータサイエンスの概念だ。1975 年に Sussman と Steele によって定義されたクロージャの定義は次のとおりだ。

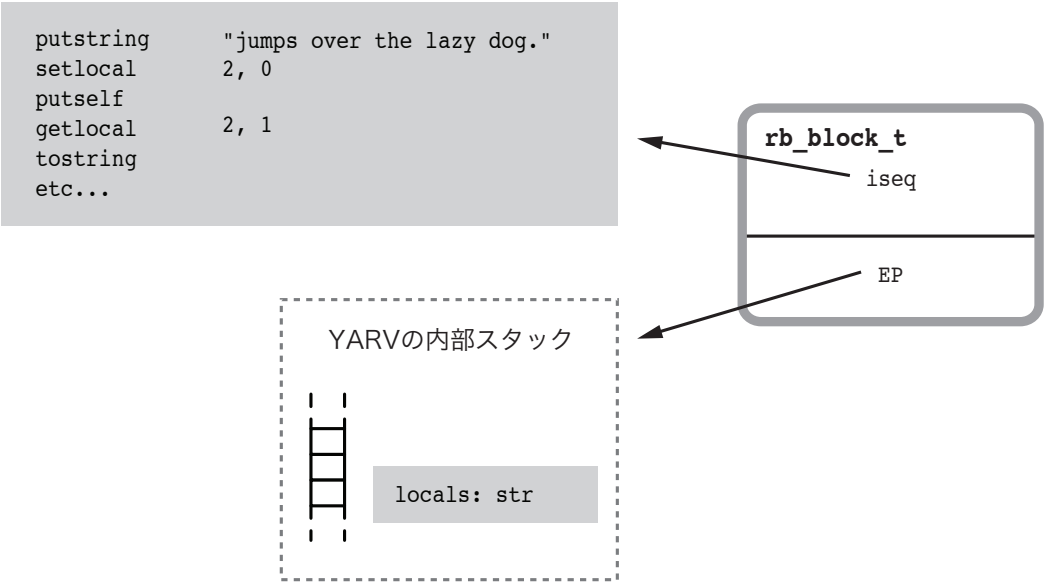
This document is licensed to kyomuwohakai@gmail.com.

この問題を解決するため、我々はクロージャ [11, 14] という概念を導入する。クロージャとは、ラムダ式と、そのラムダ式が引数に適用されるときに使われる環境の両方を持つデータ構造だ[*2]。

彼らはクロージャを次の組み合わせであると定義した。

- 「ラムダ式」——つまり、引数のセットを取る関数
- ラムダ（関数）を呼び出すときに使用される環境

それでは内部の `rb_block_t` 構造体をもう一度見てみよう。図 8-8 に再掲する。



▲ 図 8-8　ブロックは関数と関数を呼び出す際に使用される環境の組み合わせだ

この構造体は、Sussman と Steele のクロージャの定義と一致する。

- `iseq` はラムダ式へのポインタ——関数あるいはコード片
- `EP` はラムダ（関数）を呼び出した際に使われる環境へのポインタ——つまり周囲のスタックフレームへのポインタ

つまり、ブロックは Ruby におけるクロージャの実装であるといえるだろう。皮肉なことに、ブロック——Ruby をとてもエレガントでモダンなものにしている機能——は Ruby が生まれる 20 年以上も前の研究や仕事に根ざしているんだ！

[*2] Gerald J. Sussman and Guy L. Steele, Jr., “Scheme: An Interpreter for Extended Lambda Calculus” (MIT Artificial Intelligence Laboratory, AI Memo No. 349, December 1975).

This document is licensed to kyomuwohakai@gmail.com.

rb_block_t と rb_control_frame_t 構造体

Ruby 1.9 以降では、rb_block_t 構造体の定義は vm_core.h で見つけることができる。リスト 8-3 にその定義を示す。

▽ リスト 8-3　vm_core.h 内にある rb_block_t の定義

```
   typedef struct rb_block_struct {
①      VALUE self;
②      VALUE klass;
③      VALUE *ep;
④      rb_iseq_t *iseq;
⑤      VALUE proc;
   } rb_block_t;
```

iseq（④）と ep（③）に加えて、他のいくつかのメンバも確認できる。

- self（①）：ブロックが最初に参照された際の self ポインタの値で、クロージャの環境の重要な部分。Ruby はブロックの外側のコードと同じオブジェクトコンテキスト内でブロックを実行する。
- klass（②）：self と共に、Ruby はこのポインタを使用して現在のオブジェクトのクラスを追跡する。
- proc（⑤）：ブロックから Proc オブジェクトを作成した際に、Ruby はこの値を使用する。次の節で見るように、Proc オブジェクトとブロックは密接に関連している。

vm_core.h 中の rb_block_t の定義のすぐ上に、リスト 8-4 に示すとおりの rb_control_frame_t 構造体の定義を見つけることができる。

▽ リスト 8-4　vm_core.h 内にある rb_control_frame_t の定義

```
   typedef struct rb_control_frame_struct {
       VALUE *pc;                  /* cfp[0] */
       VALUE *sp;                  /* cfp[1] */
       rb_iseq_t *iseq;            /* cfp[2] */
       VALUE flag;                 /* cfp[3] */
①      VALUE self;                 /* cfp[4] / block[0] */
       VALUE klass;                /* cfp[5] / block[1] */
       VALUE *ep;                  /* cfp[6] / block[2] */
       rb_iseq_t *block_iseq;      /* cfp[7] / block[3] */
②      VALUE proc;                 /* cfp[8] / block[4] */
       const rb_method_entry_t *me;/* cfp[9] */

   #if VM_DEBUG_BP_CHECK
       VALUE *bp_check;            /* cfp[10] */
   #endif
   } rb_control_frame_t;
```

This document is licensed to kyomuwohakai@gmail.com.

この C 構造体が、self（①）から proc（②）に至るまで、rb_block_t と同じ値を含んでいることに注目してほしい。これら 2 つの構造体が同じ値を共有しているのは、Ruby が高速化のために内部的に利用する、興味深いがややこしい最適化の工夫のひとつだ。メソッド呼び出しに渡すことでブロックを最初に参照するたび、Ruby は新しい rb_block_t 構造体を作成し、現在の rb_control_frame_t 構造体から EP などの値をコピーする必要がある。しかし、2 つの構造体は同じ順番で同じ値を含んでいる（rb_block_t は rb_control_frame_t のサブセットとなっている）ので、Ruby は新しい rb_block_t 構造体を作成する必要がなく、代わりに rb_control_frame_t 構造体の共通している部分に新しいブロックポインタを設定する。つまり、新しい rb_block_t 構造体を保持するためにメモリを割り当てる代わりに、Ruby は単に rb_control_frame_t 構造体の中央へのポインタを渡すということだ。これによって、Ruby は不必要な malloc 呼び出しを回避し、ブロックの作成処理を高速化している。

実験 8-1：while ループと each にブロックを渡すのとどちらが速いか

ブロックを含む Ruby コードは、多くの場合、C のような Ruby よりも古い言語で同じことを書いた場合よりもより簡潔でエレガントなものになる。たとえば、C で 1 から 10 までの数を足していく単純な while ループを書くとリスト 8-5 のようになる。

▽ リスト 8-5　C で while ループを使って 10 までの数を足す

```
#include <stdio.h>
main()
{
  int i, sum;
  i = 1;
  sum = 0;
  while (i <= 10) {
    sum = sum + i;
    i++;
  }
  printf("Sum: %d\n", sum);
}
```

同じことを Ruby の while ループを使って書くと、リスト 8-6 のようになる。

This document is licensed to kyomuwohakai@gmail.com.

▽ リスト 8-6　Ruby で while ループを使って 10 までの数を足す

```
sum = 0
i = 1
while i <= 10
  sum += i
  i += 1
end
puts "Sum: #{sum}"
```

しかし、より Rubyist らしく書くのなら、範囲オブジェクトとブロックを使ってリスト 8-7 のようにするだろう。

▽ リスト 8-7　Ruby で範囲オブジェクトとブロックを使って 10 までの数を足す

```
sum = 0
(1..10).each do |i|
  sum += i
end
puts "Sum: #{sum}"
```

美しさはさておいて、ここでブロックを使用するためのパフォーマンス上のペナルティはあるだろうか？ 新しい rb_block_t を作成し、EP の値をコピーし、新しいスタックフレームを作成しようとすると、Ruby は大幅に遅くならないのだろうか？

まあ、C コードのベンチマークを取る必要はないだろう。Ruby を使って何をしたとしても C コードのほうが速くなることは明らかだ。代わりに、Ruby で単純な while ループを使って 1 から 10 の整数を順に足していき 55 を得るためにはどれくらいかかるかを計測してみよう。実験用のコードをリスト 8-8 に示す。

▽ リスト 8-8　while ループのベンチマークを取る（while.rb）

```
require 'benchmark'
ITERATIONS = 1000000
Benchmark.bm do |bench|
  bench.report("iterating from 1 to 10, one million times") do
    ITERATIONS.times do
      sum = 0
      i = 1
      while i <= 10
        sum += i
        i += 1
      end
    end
  end
end
```

ここでは while ループを 100 万回繰り返すのにかかる時間を計測する

This document is licensed to kyomuwohakai@gmail.com.

ために benchmark ライブラリを使用している。100 万回の繰り返し処理（ITERATIONS.times do）を制御するためにブロックを使用はしているが、次のテストでも同じようにブロックを使用する。私のラップトップ上の Ruby 2.0 を使ったところ、0.5 秒に満たない時間でこのコードを実行できる。

```
$ ruby while.rb
      user     system      total        real
      iterating from 1 to 10, one million times  0.440000   0.000000
                                                 0.440000 (  0.445757)
```

では、次にリスト 8-9 に示すブロック付き each を使ったコードの実行時間を計測しよう。

▽ リスト 8-9　ブロック呼び出しのベンチマークを取る（each.rb）

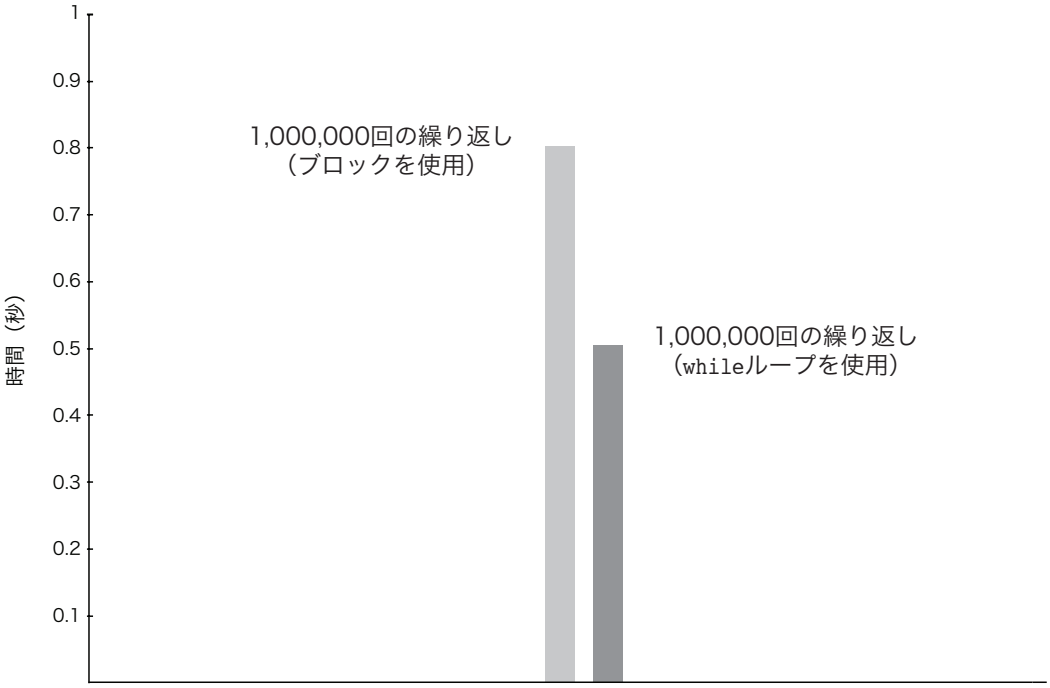
```
require 'benchmark'
ITERATIONS = 1000000
Benchmark.bm do |bench|
  bench.report("iterating from 1 to 10, one million times") do
    ITERATIONS.times do
      sum = 0
      (1..10).each do |i|
        sum += i
      end
    end
  end
end
```

今回は多少時間がかかり、100 万回の繰り返し処理に約 0.75 秒かかっている。

```
$ ruby each.rb
      user     system      total        real
      iterating from 1 to 10, one million times  0.760000   0.000000
                                                 0.760000 (  0.765740)
```

while ループによる繰り返し処理とブロック呼び出しによる繰り返し処理を 10 回ずつ行ったものを比較すると、Ruby はブロック呼び出しに約 71% ほど多くの時間を必要とすることがわかる（図 8-9 参照）。

This document is licensed to kyomuwohakai@gmail.com.



▲ 図 8-9　Ruby 2.0 はブロック呼び出しにおいて単純な while ループと比べて 71% 以上の時間がかかっている。グラフは 100 万回繰り返し処理を行った際の時間（秒）を示している

each を使用すると遅くなる理由は、Range#each メソッドはループの繰り返しごとにそのつどブロックを呼び出すか yield する必要があるからだ。ブロックの呼び出しにはかなりの量の仕事が含まれる。ブロックを yield するために、Ruby はまずそのブロック用に新しい rb_block_t 構造体を作成する必要がある。続いて、新しい rb_block_t 構造体の EP に参照元の環境を設定し、ブロックを each 呼び出しへと引き渡す。それから、ループを繰り返すごとに Ruby は YARV の内部スタックに新しいスタックフレームを作成し、ブロックのコードを呼び出す。そして、最後にブロックの EP を新しいスタックフレームにコピーする必要がある。それに比べて、単純な while ループの実行は、ループの繰り返しごとに PC（プログラムカウンタ）をリセットするだけでよいため、速い。while ループの実行はメソッドも呼び出さないし、新しいスタックフレームも新しい rb_block_t 構造体も作成しない。

71% 以上の時間がかかるとなると、パフォーマンス上の大きなペナルティであるように思える。けれど、これが重要かどうかは、あなたのやろうとしていることや while ループの文脈に依存する。もしこのループが、エンドユーザが終了を待っている時間的制約のある重要な操作の一部で、かつループの中で他に時間のかかる処理をしていないのなら、この繰り返し処理を C スタイルの while ループを使って書く価値があるかもしれない。しかし、ほとんどの Ruby アプリケーション、そしてもちろん、Ruby on Rails アプリケーションのパフォーマンスは、

This document is licensed to kyomuwohakai@gmail.com.

データベース操作やネットワーク接続などその他の要因に制約され、Ruby の実行速度は問題にならない。アプリケーション全体のパフォーマンスに、Ruby の実行速度が直接的な影響を持っていることは稀だ（もちろん、もしあなたが Ruby on Rails のような巨大なフレームワークを使っているなら、あなたが書いた Ruby コードは巨大なシステムの中のほんの小さな一部でしかない。Rails は、単純な HTTP リクエストを処理する間、あなたが書いたコードとは別に、とても多くのブロックやイテレータを使っているだろう）。

ラムダと Proc：関数を第一級市民として扱う

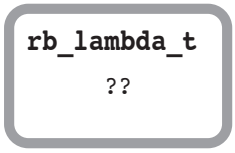
ここでは、文字列「The quick brown fox」をコンソールに表示する、より複雑な方法を見ていこう。リスト 8-10 に lambda を使った例を示す。

▽ リスト 8-10　Ruby で lambda を使う

```
① def message_function
②   str = "The quick brown fox"
③   lambda do |animal|
④     puts "#{str} jumps over the lazy #{animal}."
    end
  end
⑤ function_value = message_function
⑥ function_value.call('dog')
```

このコードを 1 つずつ丁寧に追っていこう。まず ① で message_function というメソッドを定義している。message_function 内では、② でローカル変数 str を作成している。次に、③ で lambda を呼び出し、ブロックを渡している。このブロック内では、④ で文字列「The quick brown fox」を再び表示している。しかし、③ のタイミングでは実際にはまだブロックは呼び出されていないので、message_function はすぐには文字列を表示しない。代わりに、lambda は与えられたブロックをデータ値として返し、message_function もそのままそのデータ値を返す。

コンピュータサイエンスでよく言われる表現に言い換えると、これは「関数を第一級市民として扱う」一例だ。いったん message_function から戻されると、ブロックは ⑤ でローカル変数 function_value に保存され、そして ⑥ で call メソッドを使って明示的に呼び出される。lambda キーワード――あるいはそ



▲ 図 8-10　Ruby は rb_lambda_t 構造体を使っている？ もしそうなら、その構造体は何を含んでいる？

This document is licensed to kyomuwohakai@gmail.com.

れと同等の proc キーワード――を使うと、Ruby ではこのようにブロックをデータ値に変換できる。

リスト 8-10 については、まだまだ疑問がある。lambda を呼び出したとき、一体何が起こっているのだろうか？ Ruby はどのようにブロックをデータ値に変換しているのだろう？ ブロックを第一級市民としてみなすという言葉の意味は？ message_function は rb_block_t 構造体を直接返しているのだろうか？ それとも rb_lambda_t 構造体を返しているのだろうか？ そして、rb_lambda_t に含まれる情報は一体どんなものだろうか？（図 8-10 参照）

スタック vs. ヒープメモリ

これらの質問に答えられるようになるには、まず Ruby がデータをどのように保存するのかを詳しく見ていく必要がある。内部的に、Ruby はデータを 2 つの場所に保存する。スタックとヒープだ。

前にスタックについては見た。スタックは、Ruby がプログラムの各メソッドのローカル変数や返り値、および引数を保存する場所だ。スタック上の値はメソッドが実行されている間のみ有効だ。メソッド呼び出しから返ると、YARV はスタックフレームとその中のすべての値を削除する。

メソッド呼び出しから返った後もしばらくの間必要になる可能性がある情報を保存するとき、Ruby はヒープを使用する。ヒープ内の各値は、それらが参照されている間は有効であり続ける。プログラム中の任意の変数やオブジェクトから参照されなくなると、Ruby の GC はそれを削除し、他の用途のためにそのメモリを解放する。

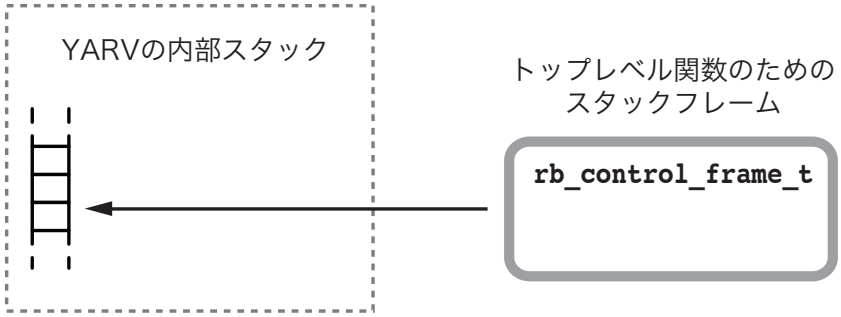
この方式は Ruby 固有のものではない。実際、Lisp や C を含む他の多くのプログラミング言語で使われている。そして、Ruby 自体も C プログラムであることを思い出してほしい。YARV のスタックは C プログラムがスタックを扱う方法を元に設計されていて、Ruby のヒープは C のヒープ実装を下敷きにしている。

スタックとヒープはもう 1 つ、重要な面で違いがある。Ruby はスタック上にデータへの参照のみを保存する――それが VALUE ポインタだ。単純な整数値やシンボル、nil や true、false といった定数値への参照は実際の値だ。しかし、他のすべてのデータ型では、VALUE は、実際の値を含んだ、RObject などの C 構造体へのポインタを指す。もし VALUE による参照のみがスタック上に保存されるなら、Ruby は実際の構造体をどこに保存するのだろうか？ それはヒープの中だ。それでは、このことをきちんと理解できるよう、例を見ていくことにしよう。

This document is licensed to kyomuwohakai@gmail.com.

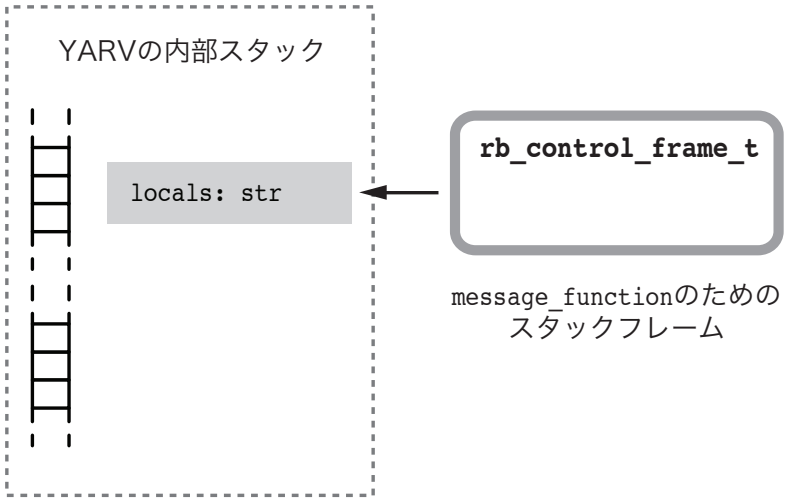
Ruby が文字列の値をどう保存するかを詳しく見る

Ruby がリスト 8-10 の文字列の値 `str` をどのように扱うかを詳しく見ていこう。まず、YARV は外側のスコープのスタックフレームを持っているけれど、まだ `message_function` を呼び出すには至っていないという状況を想像してほしい。図 8-11 に、この最初のスタックフレームを示す。



▲ 図 8-11　まっさらなスタックフレームと共に、Ruby はリスト 8-11 のコードの実行を開始する

この図では、左側に YARV の内部スタックを、右側に `rb_control_frame_t` 構造体を確認できる。今、Ruby はリスト 8-10 に示すように `message_function` を実行したところだとしよう。次に何が起きるかを図 8-12 に示す。

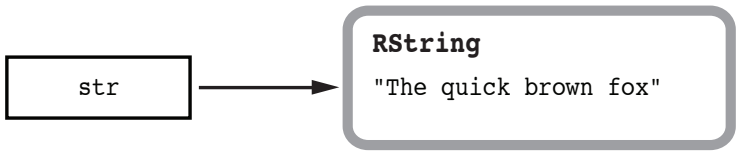


▲ 図 8-12　`message_function` を呼び出す際、Ruby は 2 つめのスタックフレームを作成する

Ruby はローカル変数 `str` を `message_function` から使われる新しいスタックフレームに保存する。変数 `str` と、Ruby がその中に文字列「quick brown fox」をどのように格納するかについて、詳しく見ていこう。Ruby は、プログラム中の各オブジェクトを `RObject` という C 構造体に格納する。各配列は `RArray` とい

This document is licensed to kyomuwohakai@gmail.com.

う C 構造体、各文字列は RString という C 構造体にそれぞれ格納する。文字列「quick brown fox」が RString に保存されることを図 8-13 に示す。



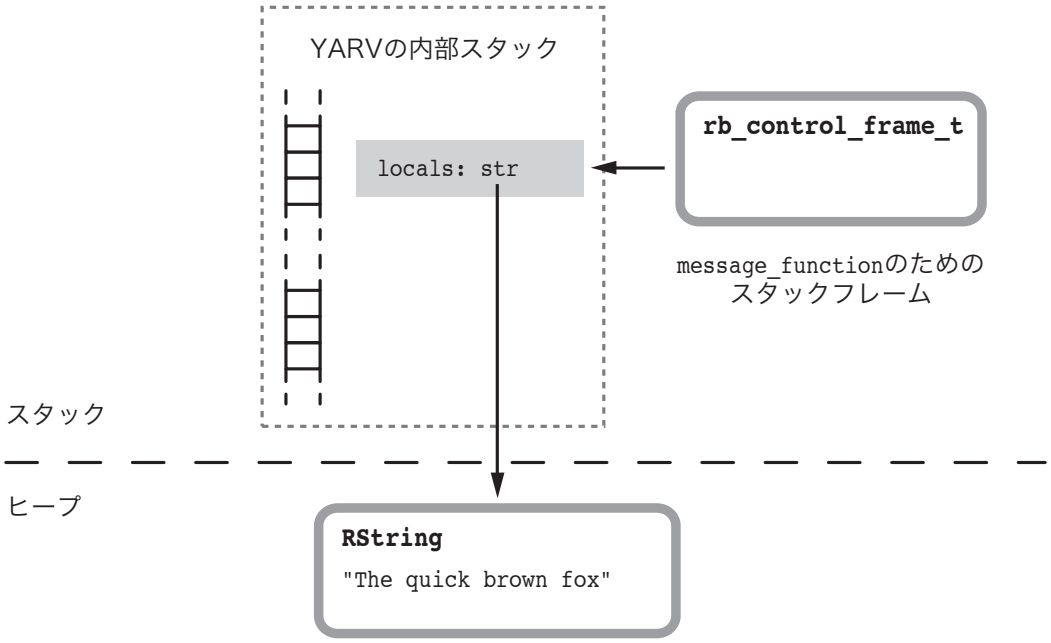
▲ 図 8-13　Ruby は文字列の値を保存するために RString 構造体を使う

実際の文字列を含む構造体を図の右側に、文字列への参照を図の左側に示している。Ruby は、YARV スタック上に文字列の値（または任意のオブジェクト）を保存したとき、実際にはスタック上には文字列への参照だけを置く。実際の文字列を含む構造体は、代わりにヒープ内に保存される。図 8-14 にこれを示す。

ヒープ内の特定のオブジェクト、または値を参照しているポインタがなくなると、Ruby はそれを次の GC 実行時に解放する。これを実証するため、lambda を全く呼び出さず、逆に変数 str を保存した後、すぐに nil を返すようにしてしまうとする。リスト 8-11 にこれを示す。

▽ リスト 8-11　このコードは lambda を呼び出さない

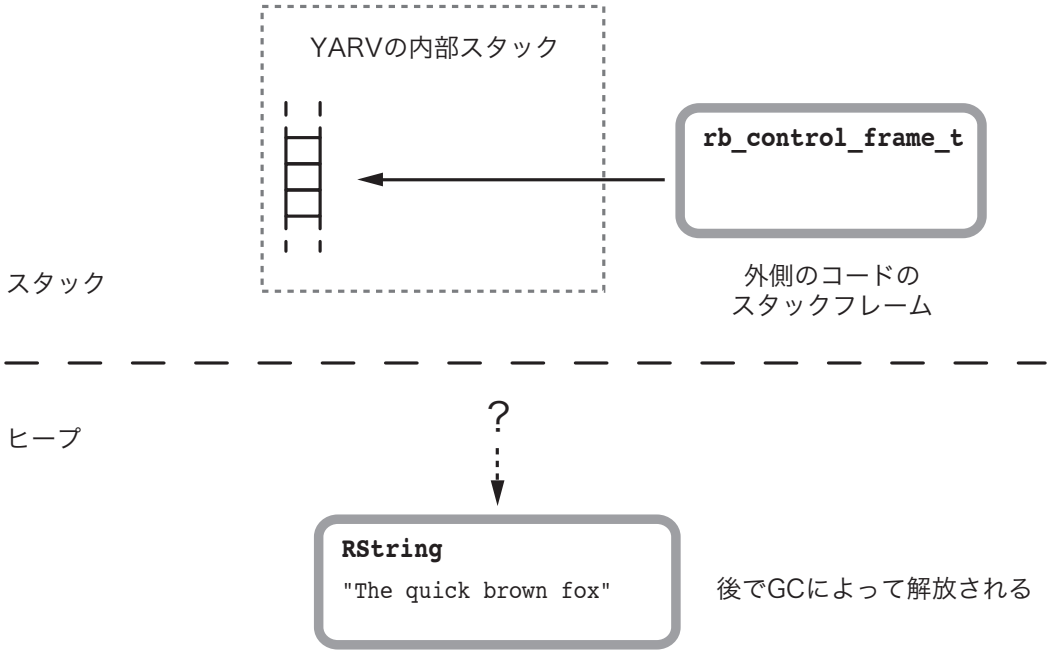
```
def message_function
  str = "The quick brown fox"
  nil
end
```



▲ 図 8-14　スタック上の値 str は、ヒープ内に保存された RString 構造体への参照だ

This document is licensed to kyomuwohakai@gmail.com.

message_function の呼び出しが終了すると、図 8-15 に示すように、YARV は str を（ここに保存された、他の一時的な値と同様に）単にスタックからポップし、元のスタックフレームに戻る。



▲ 図 8-15　もう RString 構造体への参照はない

図を見るとわかるように、「quick brown fox」文字列を含む RString 構造体への参照はもはや存在しない。Ruby の GC は、ここでいう「quick brown fox」文字列のように、ヒープ内の参照されていない値を見つけるように設計されている。GC システムはこうした値を見つけた後、それらの孤立した値を解放し、ヒープにメモリを返却する。

Ruby はラムダをどう作るか

ヒープと、Ruby がヒープをどう使うかについての理解が少し深まったところで、いよいよラムダについて詳しく学ぶ準備が整った。少し前で、「関数を第一級市民として扱う」というフレーズを使った。これは、Ruby では関数（コード）をデータ値として扱い、変数に保存したり、引数として渡したりできることを意味していた。Ruby はこのアイデアをブロックを使って実装している。

lambda（または proc）キーワードは、ブロックをデータ値に変換する。でもここで、ブロックは Ruby におけるクロージャの実装だということを思い出してほしい。これはつまり、新しいデータ値が、ブロックのコードと参照元の環境の両方をどこかに含んでいなければならないということを意味している。

このことを確認するため、リスト 8-10 (p. 226) に戻って、lambda を使っている部分に目を向けてみよう。リスト 8-12 に、リスト 8-10 のコードを再び示す。

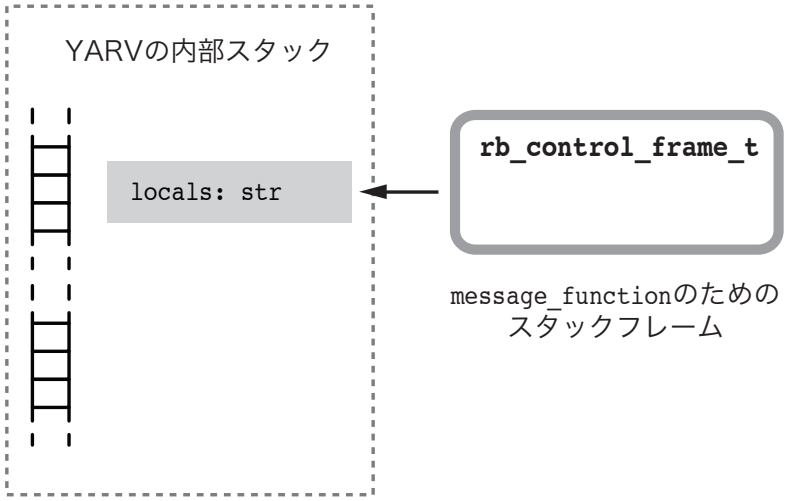
This document is licensed to kyomuwohakai@gmail.com.

▽ リスト 8-12　Ruby で lambda を使う（リスト 8-10 の再掲）

```
   def message_function
①    str = "The quick brown fox"
②    lambda do |animal|
③      puts "#{str} jumps over the lazy #{animal}."
     end
   end
   function_value = message_function
④  function_value.call('dog')
```

④ でラムダ（ブロック）を呼び出したときに、③ のブロック内の puts 命令が、message_function 内の ① で宣言された変数 str にアクセスできることに注目してほしい。どうしてこんなことができるのだろうか？ そういえばさっき、RString 構造体への参照を持つ str が、message_function から返ったとき、どのようにスタックからポップされるかを見た！ 言うまでもなく、lambda を呼び出すと、str の値はしっかり生きていて、ブロックはその値に後からアクセスできる。

lambda を呼び出すと、Ruby は現在の YARV スタックフレーム全体の内容をヒープ内にコピーする。RString 構造体もそこに含まれている。たとえば、図 8-16 はリスト 8-12 の ① で、message_function を開始した直後に YARV スタックがどのように見えるかを示している（話を簡単にするため、RString 構造体はここでは見せていない。けれど、RString 構造体もヒープ内に保存されていることを覚えておいてほしい）。



▲ 図 8-16　message_function を呼び出す際、Ruby は 2 つめのスタックフレームを作成する

次に、リスト 8-12 は ② で lambda を呼び出す。図 8-17 に、lambda を呼び出すと Ruby の中で何が起きるのかを示す。

This document is licensed to kyomuwohakai@gmail.com.

破線の下にある水平方向のスタックのアイコンは、Ruby がヒープ内に message_function 用のスタックフレームを新たにコピーすることを示している。これは str の RString 構造体への第 2 の参照であり、message_function から返っても Ruby は str の RString 構造体を解放しないことを意味している。

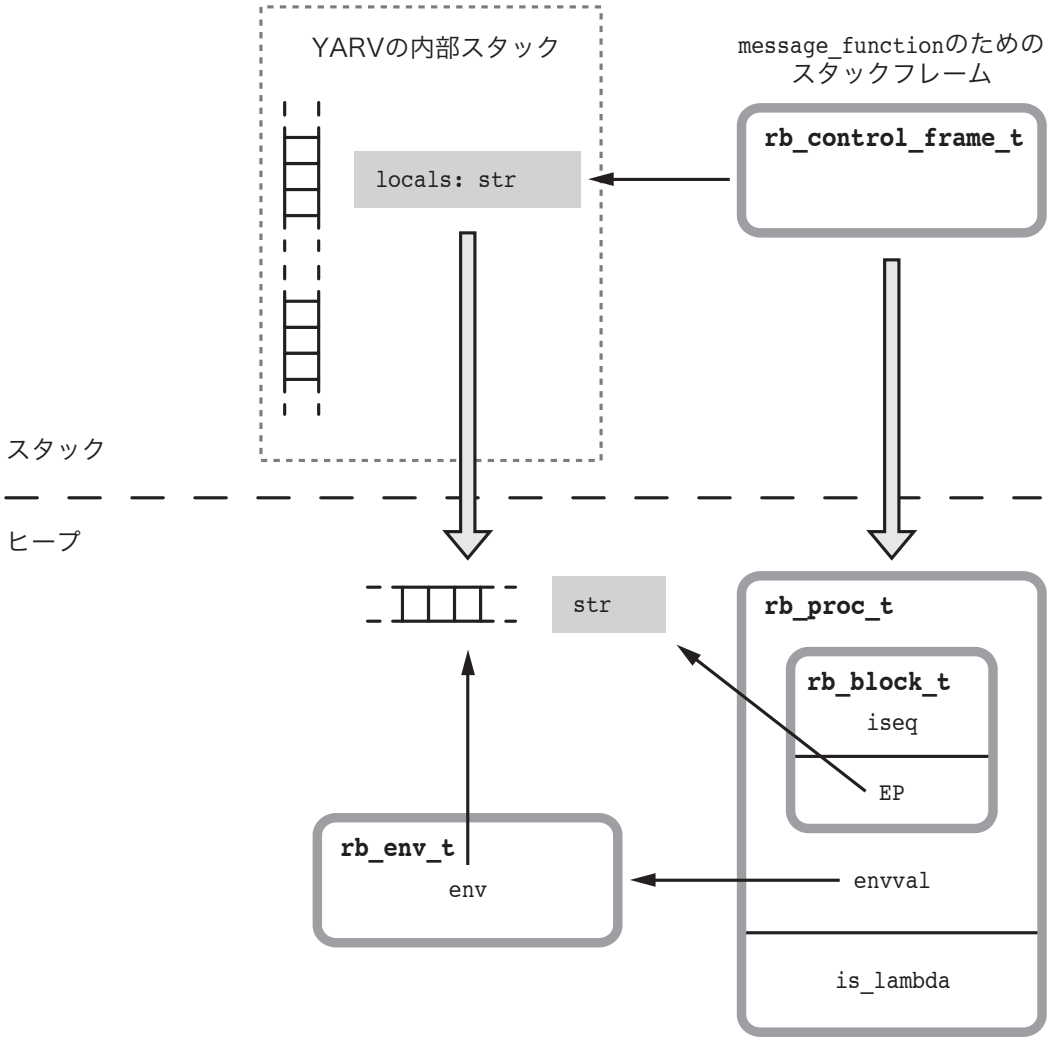
実際には、スタックフレームのコピーと一緒に、Ruby は次の 2 つの新しいオブジェクトをヒープ内に作成する。

- 内部環境オブジェクト。図の左下にある rb_env_t という C 構造体で表現される。本質的には、これはスタックのヒープコピー用のラッパーだ。第 9 章で説明するが、Binding クラスを使用することでプログラム内からこの環境オブジェクトに間接的にアクセスできる。
- Proc オブジェクト。rb_proc_t 構造体で表現される。これは lambda キーワードからの実際の返り値であり、message_function から返されている値でもある。

新しい Proc オブジェクト、rb_proc_t 構造体が、iseq や EP を持つ rb_block_t 構造体を中に含んでいることに注目してほしい。Proc オブジェクトを、ブロックを包む Ruby オブジェクトの一種だと考えてほしい。通常のブロックと同様、Proc はブロックのコードと参照元の環境を、クロージャとして保持し続ける。ヒープにコピーされた新しいスタックフレームをポイントするため、Ruby はこのブロックに EP を設定する。

Proc オブジェクトが is_lambda という内部値を含んでいることにも注目してほしい。この例では、lambda キーワードを使って Proc オブジェクトを作成しているので、この値は true に設定される。もし、proc キーワードを使用するか、単に Proc.new して Proc オブジェクトを作成していたなら、is_lambda の値は false に設定されることになる。本質的には、Proc オブジェクトとラムダは同じものだと考えてしまってよい。しかし、Ruby はこのフラグを使って、この 2 つの間にわずかな動作の差を作り上げている。

This document is licensed to kyomuwohakai@gmail.com.



▲図 8-17　lambda を呼び出すと、Ruby は現在のスタックフレームをヒープにコピーする

Ruby はラムダをどう呼び出すか

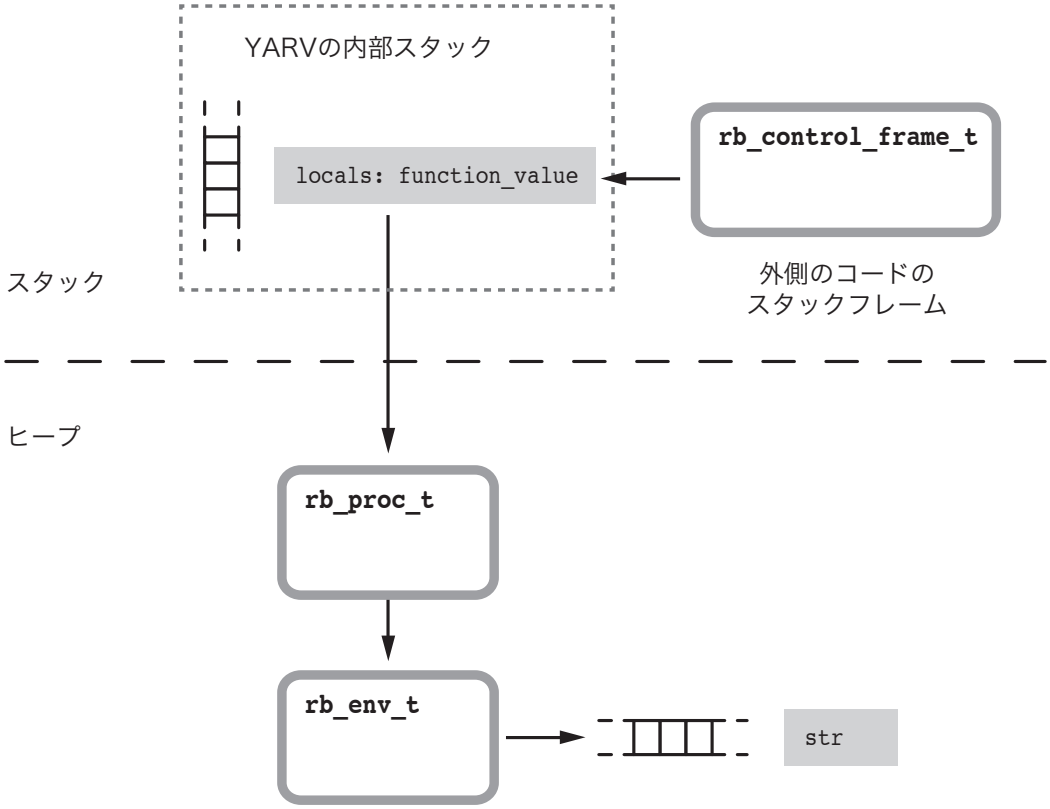
リスト 8-13 に示したラムダの例に戻ろう。

▽リスト 8-13　Ruby で lambda を使う（リスト 8-10 の再掲）

```
  def message_function
    str = "The quick brown fox"
    lambda do |animal|
      puts "#{str} jumps over the lazy #{animal}."
    end
  end
① function_value = message_function
② function_value.call('dog')
```

This document is licensed to kyomuwohakai@gmail.com.

① で message_function から返ると、一体何が起こるのだろうか？ 返り値がラムダ、または Proc オブジェクトであるため、スタックフレーム内に保存されていたラムダへの参照が、外側のスコープのローカル変数 function_value に保存される。これは、Ruby が Proc オブジェクト、内部環境オブジェクト、変数 str を解放するのを防ぐ。これによって、ヒープ内のこれらの値すべてを参照するポインタが存在することになる（図 8-18 参照）。



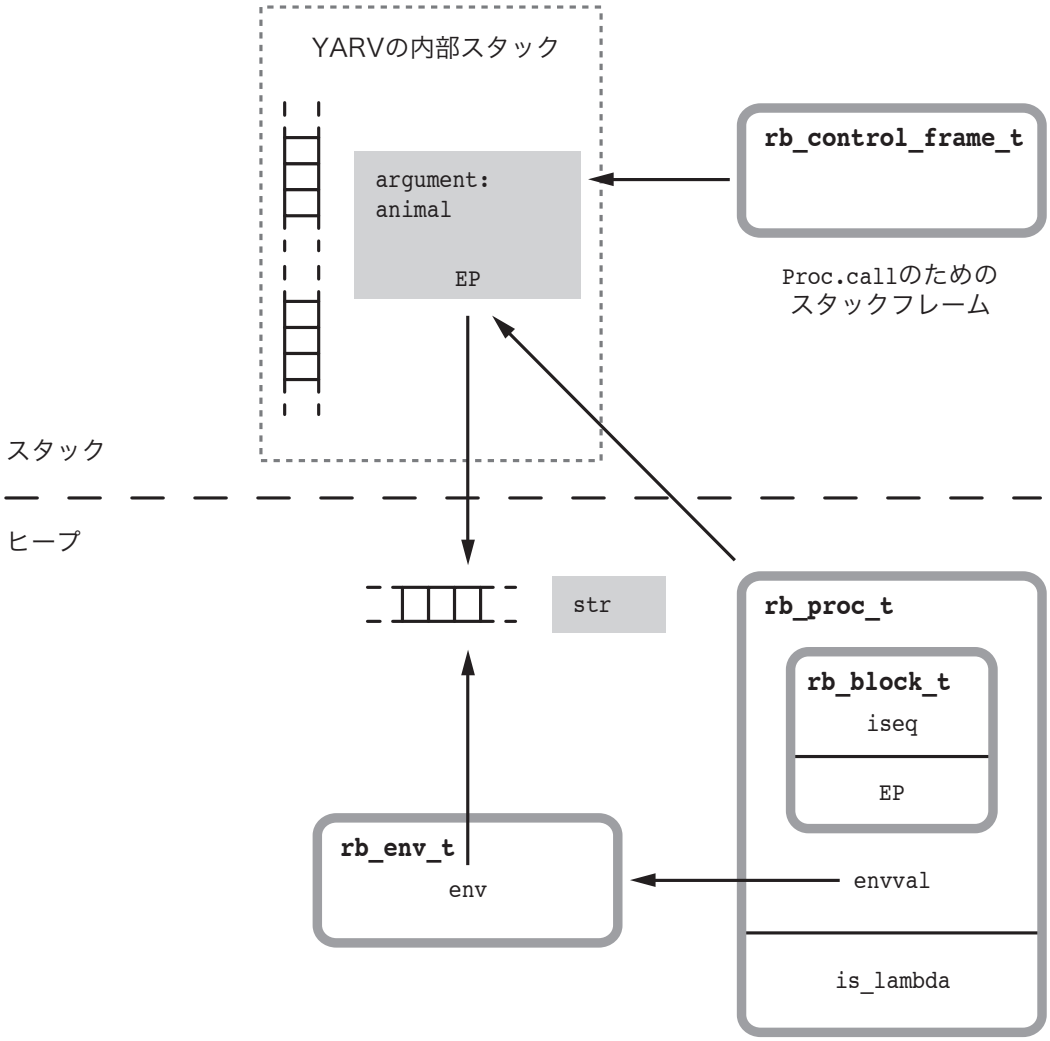
▲図 8-18　いったん message_function から返ると、周囲のコードは Proc オブジェクトへの参照を保持する

② で Ruby が Proc オブジェクトの call メソッドを実行すると、Proc オブジェクトは同様に保持しているブロックを実行する。図 8-19 に、ラムダまたは Proc オブジェクトの call メソッドを呼び出した際に、Ruby の中で何が起こるかを示す。

任意のブロックを呼び出したときと同様に、Ruby は Proc オブジェクト内のブロックを呼び出すと、新しいスタックフレームを作成し、EP をブロックの参照元の環境に設定する。しかし、その環境は以前にヒープにコピーされたスタックフレームのコピーだ。新しいスタックフレームはヒープにポイントした EP を含んでいる。この新しい EP によって、ブロック呼び出しは message_function 内で定義された値 str にアクセスできる。図 8-19 に、他の任意のメソッドやブロッ

This document is licensed to kyomuwohakai@gmail.com.

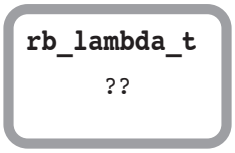
クの引数と同様に、Proc オブジェクトの引数 animal が新しいスタックフレームに保存されていることを示す。



▲図 8-19　Proc オブジェクトを呼び出すと、いつものように新しいスタックフレームを作成し、ヒープの参照元の環境を示すように EP を設定する

Proc オブジェクト

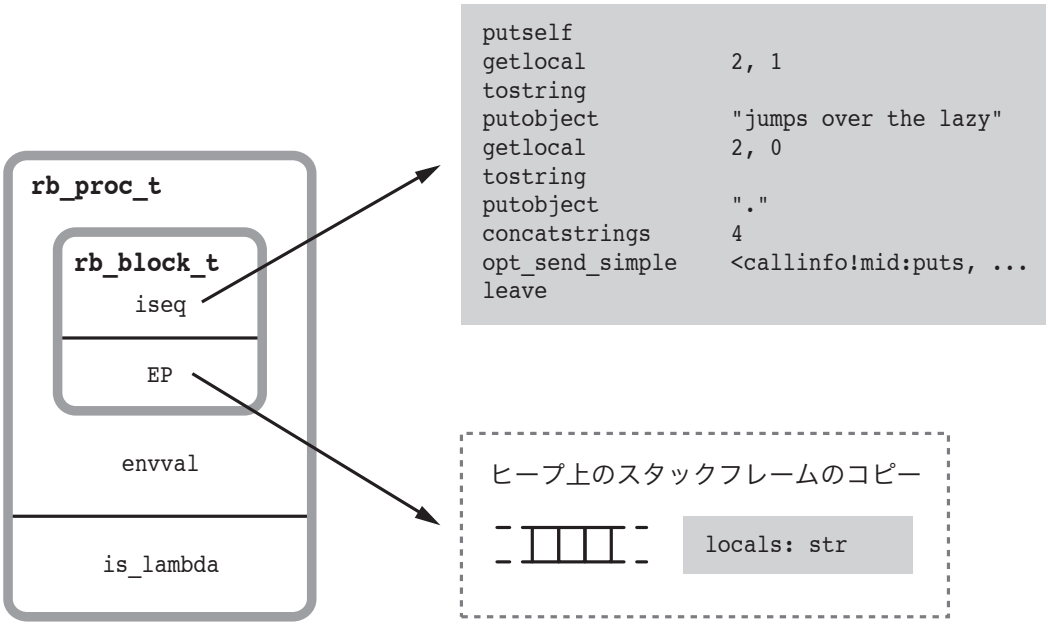
ここまで、Ruby が実際には rb_lambda_t という構造体は持たないということを見てきた。つまり、図 8-20 のような構造体は実際には存在しない。



▲図 8-20　Ruby は実際には rb_lambda_t という構造体は使わない

This document is licensed to kyomuwohakai@gmail.com.

その代わり、この例において、Ruby の lambda キーワードは Proc オブジェクトを作成した。この Proc オブジェクトというのは、実際には lambda や proc キーワードに渡したブロックのラッパーだ。Ruby は Proc オブジェクトを rb_proc_t 構造体を使って表現する。図 8-21 にこれを示す。



▲ 図 8-21　Ruby の Proc オブジェクトは、関数と参照元の環境へのポインタを含むクロージャだ

これはクロージャだ。関数と関数が参照している環境、もしくは関数が作られた環境を含んでいる。環境とは、ヒープ内に保存され存在し続ける、スタックフレームのコピーだ。

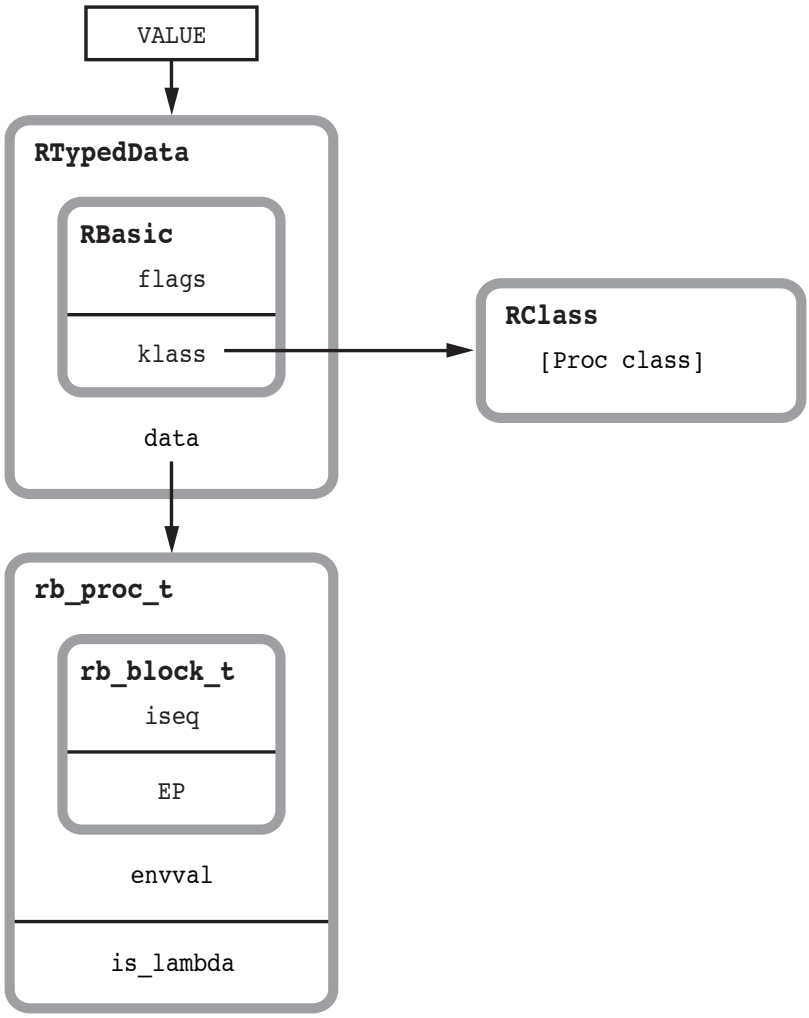
Proc は Ruby オブジェクトだ。RBasic 構造体を持ち、他のオブジェクトと同じ情報を含んでいる。オブジェクト関連情報を保存するために、Ruby は Proc オブジェクトのインスタンスを表現する rb_proc_t と一緒に、RTypedData という構造体を使用する。図 8-22 にこれらの構造体がどう一緒に動くかを示す。

RTypedData のことを、C のデータ構造を包んだ Ruby オブジェクトを作成するために使用するトリックの一種と考えるかもしれない。この場合、rb_proc_t 構造体の単一のコピーを表現する Proc クラスのインスタンスを作成するために、Ruby は RTypedData を使用する。RTypedData 構造体は、すべての Ruby オブジェクトと同じように RBasic 情報を含んでいる。

flags　Ruby が追跡する必要のある特定の内部情報。

klass　何のインスタンスであるかを示す Ruby クラスへのポインタ。この例では Proc クラスになる。

This document is licensed to kyomuwohakai@gmail.com.

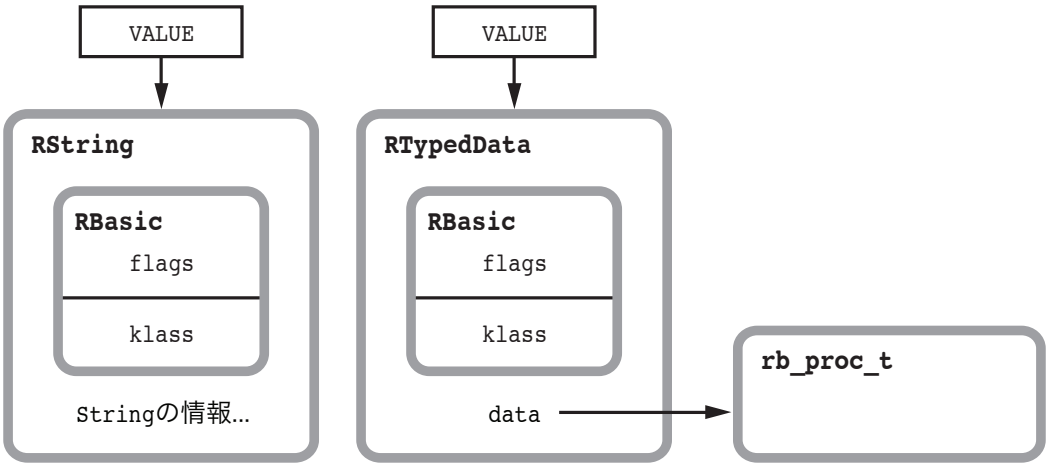


▲図 8-22　Ruby は RTypedData 構造体に Proc オブジェクトに関連する情報を保存する

図 8-23 は Ruby が Proc オブジェクトをどのように表現するかを、別の形で表している。RString 構造体のすぐ右隣が Proc オブジェクトだ。

Ruby が文字列の値と Proc を同じように処理することに注目してほしい。文字列と同じように、Proc オブジェクトは変数に保存され、関数呼び出しに引数として渡すことが可能だ。Ruby は Proc を参照したり変数内に保存したりする際はいつでも、Proc への VALUE ポインタを使う。

This document is licensed to kyomuwohakai@gmail.com.



▲ 図 8-23　Proc と文字列の比較

実験 8-2：ラムダを呼び出した後でローカル変数を変更する

リスト 8-10 (p. 226)からリスト 8-13 (p. 233)までを通して、lambda 呼び出しがどのようにヒープに現在のスタックフレームをコピーするかを見てきた。ここでは、少しだけ違った例を示す。リスト 8-14 は、基本的にはこれまでの例と同じだ。しかし、② で lambda を呼び出した後に str を変更している点が異なっている。

▽ リスト 8-14　lambda は str のどのバージョンをヒープにコピーするか？（modify_after_lambda.rb）

```
   def message_function
     str = "The quick brown fox"
①    func = lambda do |animal|
       puts "#{str} jumps over the lazy #{animal}."
     end
②    str = "The sly brown fox"
     func
   end
   function_value = message_function
③  function_value.call('dog')
```

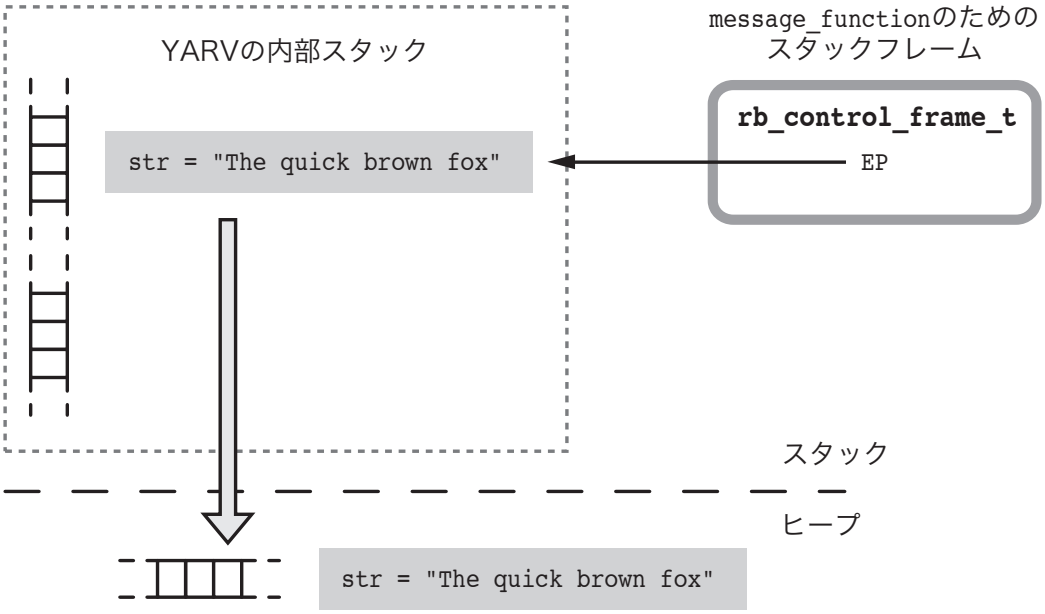
② で文字列を「The sly brown fox」に変更する前に、① で lambda を呼び出しているので、Ruby は元の str の値を含む内容で、ヒープにスタックフレームをコピーすべきだ。つまり、③ で lambda を呼び出すと、元の文字列「The quick brown fox」が表示されるべきだ。しかし、これを実行すると結果は次のようになる。

This document is licensed to kyomuwohakai@gmail.com.

```
$ ruby modify_after_lambda.rb
The sly brown fox jumps over the lazy dog.
```

何が起こったんだろう？ Ruby はなんらかの方法で str の新しい値「The sly brown fox」をヒープにコピーした。おかげで、③ でラムダを呼び出すときに新しい値にアクセスできたのだ。

Ruby がこれをどう処理したかを探るため、lambda を呼び出すときに何が起こるかをもっと詳しく見ていこう。図 8-24 に、リスト 8-14 の str を含んでいるスタックフレームを、Ruby がどうやってヒープにコピーするかを示す。



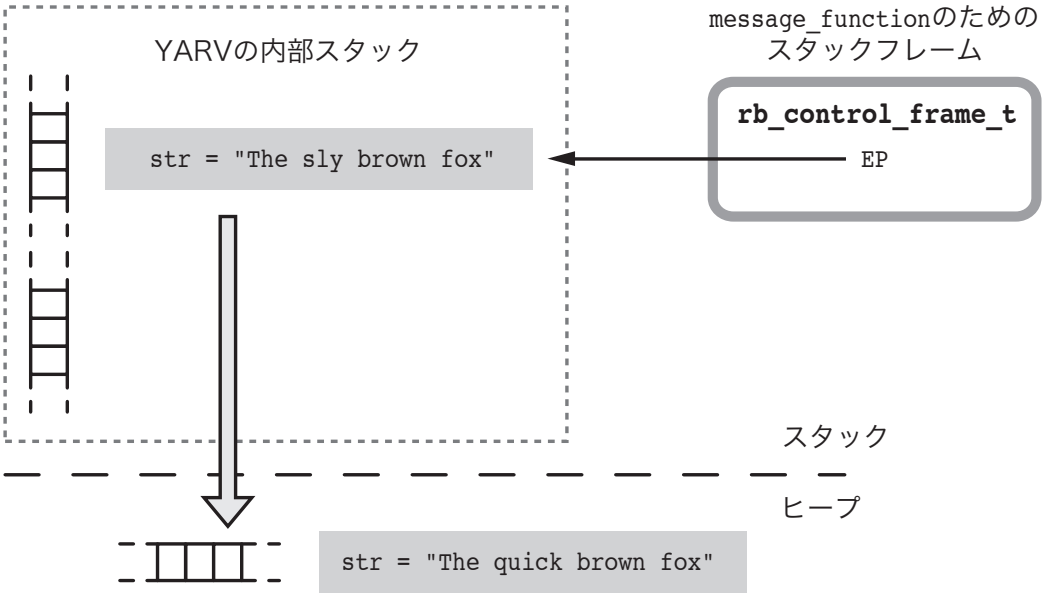
▲図 8-24　lambda を呼び出すと、Ruby はヒープにスタックフレームをコピーする

このコピーが行われたのち、リスト 8-14 の ② のコードは str を文字列「sly fox」に変更する。

```
str = "The sly brown fox"
```

lambda を呼び出した際に、Ruby はスタックフレームをコピーした。そのため、新しいラムダのコピーではなく、コピー元の str を編集しているはずだ（図 8-25 参照）。

This document is licensed to kyomuwohakai@gmail.com.

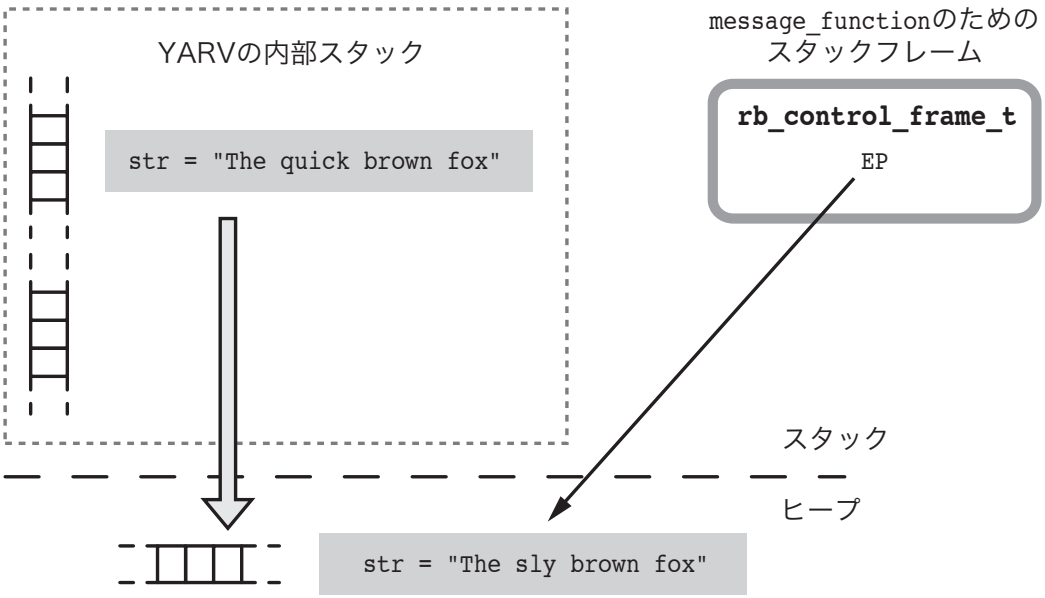


▲図 8-25　ヒープにコピーが終わった後も、Ruby は元のスタックフレームを使い続ける？

ヒープ上の新しい文字列のコピーは編集されないままで残ったはずだ。そして、後でラムダを呼び出した際には、編集された「sly fox」ではなく、元の文字列「quick fox」を返すはずだった。では、Ruby はどのようにして、`lambda` によって作成されたスタック上の新しいコピーを編集できるのだろうか？

実際には、Ruby はヒープにスタック（新しい `rb_env_t` 構造体、あるいは内部環境オブジェクト）の新しいコピーを作成すると、コピーしたところを指すよう `rb_control_frame_t` 構造体中の `EP` をリセットする。図 8-26 に、スタックフレームのヒープコピーを作成した後、Ruby がどうやって `EP` をリセットするかを示す。

This document is licensed to kyomuwohakai@gmail.com.



▲図 8-26　Ruby はスタックフレームのヒープへのコピーが完了した後 EP をリセットする

ここでの違いは、EP が下にあるヒープを指していることだ。リスト 8-14 の ②で str = "The sly brown fox"を呼び出すと、Ruby は新しい EP を使用してスタック上の元の値ではなくヒープ内の値にアクセスする。図 8-26 の下のヒープ内に「The sly brown fox」があることに注目してほしい。

同じスコープでラムダを 1 回以上呼び出す

lambda キーワードの、興味深いもう 1 つの振る舞いは、ruby が複数回スタックフレームをコピーするのを回避することだ。これはリスト 8-15 で確認できる。

▽リスト 8-15　同じスコープで lambda を 2 回呼び出す

```
i = 0
increment_function = lambda do
  puts "incrementing from #{i} to #{i+1}"
  i += 1
end
decrement_function = lambda do
  i -= 1
  puts "decrementing from #{i+1} to #{i}"
end
```

このコードでは、両方のラムダでメインスコープ内のローカル変数 i を操作することを期待している。

もし Ruby が、それぞれの lambda 呼び出しに対して、スタックフレームのコピーを別々に作成したなら、それぞれの関数は異なる i のコピーを操作すること

This document is licensed to kyomuwohakai@gmail.com.

になるだろう。リスト 8-16 に示す例を見てみよう。

▽ リスト 8-16　リスト 8-15 で生成されたラムダを呼び出す

```
increment_function.call
decrement_function.call
increment_function.call
increment_function.call
decrement_function.call
```

もし Ruby がラムダ関数ごとに i の別々のコピーを使っていたなら、先のコードはリスト 8-17 に示すような出力を生成するはずだ。

▽ リスト 8-17　生成された各 lambda の呼び出しがスタックフレームの独自のコピーを持つ場合に期待する出力

```
Incrementing from 0 to 1
Decrementing from 0 to -1
Incrementing from 1 to 2
Incrementing from 2 to 3
Decrementing from -1 to -2
```

けれど、実際にはリスト 8-18 のような結果となる。

▽ リスト 8-18　ラムダ関数は同じヒープコピーとスタックを共有するので、リスト 8-16 の実行結果はこのようになる

```
Incrementing from 0 to 1
Decrementing from 1 to 0
Incrementing from 0 to 1
Incrementing from 1 to 2
Decrementing from 2 to 1
```

通常はこちらが期待している結果だろう。ラムダに渡す各ブロックは、親スコープの同じ変数にアクセスする。EP が既にヒープを指すかどうかをチェックすることで、Ruby はこれを実現する。EP が既にヒープを指していた場合（リスト 8-15 の 2 回目の lambda の呼び出しのような場合）は、Ruby は 2 つめのコピーを作成しない。単に同じ rb_env_t 構造体を 2 つめの rb_proc_t 構造体に再利用する。最終的に、両方のラムダはヒープ上の同じスタックフレームのコピーを使用する。

This document is licensed to kyomuwohakai@gmail.com.

まとめ

第 3 章では、ブロックを呼び出すと、YARV はメソッドを呼び出したときと同様に、新しいスタックフレームを作成するということを学んだ。一見すると、Ruby のブロックは、引数を渡して呼び出せる特別な種類のメソッドのように見える。けれど、この章で見てきたとおり、ブロックは見た目ほど単純ではない。

この章では、`rb_block_t` 構造体を詳しく見ることで、Ruby において、ブロックがクロージャというコンピュータサイエンスの概念をどのように実装したのかを知った。ブロックは、関数と関数を呼び出した際に利用する環境の組み合わせだ。そして、ブロックが奇妙な 2 つの人格を持つことを学んだ。ブロックはメソッドと同じようなものだ。けれど、ブロックは呼び出し元のメソッドの一部にもなる。この 2 つの役割をシンプルに実現できる Ruby の構文は、Ruby における最も美しくエレガントな機能のひとつといえる。

その後、Ruby が `lambda` キーワードを使って、関数（コード）をどのように第一級市民として扱うかを見た。`lambda` キーワードは、ブロックをデータ値へと変換し、渡したり、保存したり、再利用できるようにする。そして、スタックとヒープの違いを確認した後、Ruby における `lambda` や `Proc` オブジェクトの実装方法を調査した。`lambda`（`Proc` オブジェクト）を呼び出すと、Ruby はスタックフレームをヒープにコピーする。そして、`lambda` によってデータ値となったブロックを呼び出すと、Ruby はそれを再利用する。最後に、Proc オブジェクトがどのようにコードを Ruby のデータオブジェクトとして表現するかを説明した。

This document is licensed to kyomuwohakai@gmail.com.